The MABEL Teleoperation Server: A Hardware-Agnostic Bridge Between Mixed-Reality Operators and a Whole-Body Simulated Robot

MABEL project — `server/`

November 4, 2026

Abstract

The MABEL teleoperation server is the network and control hub that lets an Apple Vision Pro headset, an iPhone app, or any future controller drive the MuJoCo-simulated MABEL humanoid (or, with no protocol change, the real robot) and watch its cameras in real time. It exposes a single, deliberately minimal two-port contract: operator intent flows in over one WebSocket (`:9090/teleop`) carrying typed JSON `TeleopFrame`s, and the robot's eye/wrist cameras flow back over plain HTTP MJPEG (`:8080/camera/*`) that the headset renders directly in a `WKWebView`. The core of the system is the *bridge* (`sim_teleop_bridge.py`): a single process that adapts headset hand poses into the shared whole-body controller, steps MuJoCo under a real-time accumulator, and publishes the robot's configuration to a memory-mapped file. A second process (`sim_camera_server.py`) mirrors that configuration and renders the cameras on its own core, so physics and video never contend. We document the wire protocol, the threading and decoupling design, the real-time control loop, and the weighted camera-render scheduler, and we report the measured throughput of each: with shadows disabled in the camera feeds (the default, which removes a ≈ 32 ms per-frame shadow pass and nearly doubles the render budget to ≈ 24 frames/s), the head camera sustains ≈ 12 fps and each wrist ≈ 6 fps, at $\approx 1.0\times$ real-time physics with a 4 ms timestep.

Keywords: teleoperation, whole-body control, WebSocket, MJPEG, MuJoCo, mixed reality, Apple Vision Pro, hand retargeting, real-time simulation.

1 Overview

The `server/` directory is the Mac-resident hub that closes the teleoperation loop between a mixed-reality operator and the MABEL robot. It was built around one design commitment: **the transport contract is hardware-agnostic and client-agnostic.** Every client speaks the same two endpoints, and a single running server feeds all of them at once, so the Vision Pro app, the iPhone app, and a keyboard or game controller are interchangeable front-ends. Swapping the simulated robot for the physical one is a server-side change only; the apps connect unchanged.

1. **Operator → robot.** The client streams `TeleopFrame`s (a head transform plus, per hand, an anchor transform and 27 named joint transforms in the ARKit world frame) to `ws://<mac>:9090/teleop`. The bridge adapts each frame, runs clutch-based hand retargeting into a whole-body inverse-kinematics controller, and steps the physics.
2. **Robot → operator.** The robot's simulated stereo head (ZED) and wrist cameras are rendered off-screen and served on two transports fed by the same JPEGs: *latest-frame-wins UDP* on `:9091`, the headset's primary minimum-latency path (Section 2.6), and

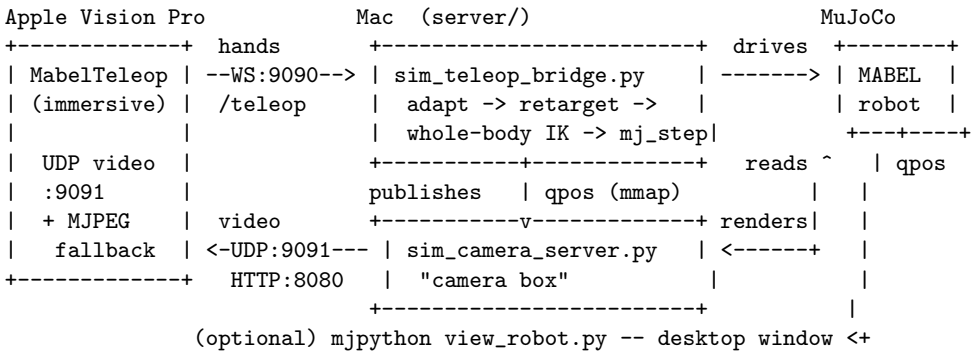


Figure 1: System architecture. One WebSocket carries operator intent in; latest-frame-wins UDP (`:9091`, with HTTP MJPEG `:8080` as the automatic fallback) carries the robot's cameras out. Physics (bridge) and rendering (camera box) are separate processes communicating only through a memory-mapped `qpos` snapshot, so they run on independent cores.

`multipart/x-mixed-replace` MJPEG at `http://<mac>:8080/camera/*/stream.mjpg`, the universal path (browser console, iOS app, and the headset's automatic fallback when a network blocks UDP).

The codebase splits into three layers, each in its own module so the sim and the real robot can share logic (Fig. 1): a *transport* layer (`wire_protocol.py`, `mjpeg_server.py`, `config.py`); an *adaptation* layer (`teleopframe_adapter.py`) that translates the headset's hand model into the controller's expected format; and the *bridge* itself (`sim_teleop_bridge.py`) plus the decoupled camera box (`sim_camera_server.py`). The whole-body control math is *not* in this directory: it lives in the shared `controller/mabel` package so the desktop sim viewer and this server can never drift apart. A simpler entry point, `main.py`, serves a webcam as the cameras and only logs poses — it is the bring-up harness that predates the sim wiring.

2 Method

2.1 Wire protocol

All messages are a two-layer JSON envelope that mirrors the Swift `Codable` types exactly (camelCase field names), so the Python server and the Swift client share one schema (`wire_protocol.py`):

$$\{\ \texttt{"type":}\ \ \texttt{<WireMessageType>, "payload":}\ \ \{\ \texttt{...}\ \}\ \}.$$

The message types form the full vocabulary of the contract: `teleop_frame` and `joint_command` (operator → robot intent), `robot_state` (robot → operator telemetry), `control_mode` and `reset` (mode/lifecycle), and `hello`/`ping`/`pong` for liveness. The decoder validates the type, rejects unknown paths at connect time, and is stdlib-only so it can be imported into a ROS2 node without extra dependencies.

The principal message, `TeleopFrame`, carries a monotonically increasing `sequence`, a `timestamp`, a `HeadPose`, an optional left and right `HandPose`, a `mode` (`Arms & Hands` or `Base Driving`), and an optional `navJoystick`. Every pose is a row-major homogeneous transform stored as 16 floats,

$$T = \begin{bmatrix} R & p \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3), \qquad R \in SO(3),\ p \in \mathbb{R}^3,$$

in the ARKit world frame (x right, y up, $-z$ forward). A `HandPose` is an `anchorTransform` plus a list of named joint `localTransforms`; the world pose of joint j is the composition

$$T_j^{\text{world}} \;=\; T_{\text{anchor}}\, T_j^{\text{local}}. \tag{1}$$

The `navJoystick` fields $(\ell_x, \ell_y, r_x, r_y)$ are normalised to $[-1, 1]$: the left stick translates the mobile base (ℓ_y forward/back, ℓ_x strafe) and the right stick yaws (r_x) and lifts (r_y).

2.2 Hand adaptation

The headset and the controller speak different hand dialects, so the bridge inserts a thin adapter (`teleopframe_adapter.py`). The controller's `Retargeter` expects a MediaPipe-style list of $SE(3)$ transforms per hand and only ever reads three indices: 0 = wrist, 4 = thumb tip, 8 = index tip. The adapter computes each of those world transforms via Eq. (1), packs them into a 9-element list (length ≥ 9 so the pinch-grasp test does not bail out), and exposes a duck-typed object with `.head`, `.left`, `.right`, `.timestamp` — identical to the sim's native `AVPFrame`. The frame is passed through in the ARKit frame; the ARKit→robot basis change lives in the controller's config, so the mapping is defined in exactly one place.

2.3 The bridge: threading and the real-time loop

The bridge runs the entire pipeline in one process with two threads, a split forced by macOS: the GL context for off-screen rendering must live on the main thread, so

- the **main thread** owns MuJoCo physics, the controller, and (optionally) the camera render; and
- a **network thread** hosts a single `asyncio` loop running both the WebSocket teleop server and, when rendering in-process, the MJPEG HTTP server.

The two threads communicate through a one-slot `deque` holding the newest `TeleopFrame` (the network thread appends, the sim thread reads), plus `threading.Events` for recalibration and reset. Only the freshest input frame is ever consumed; stale frames are discarded rather than queued, which is the single most important latency decision in the system.

Real-time accumulator. The control loop must advance simulated time to match wall-clock time even though a render call can block the thread for tens of milliseconds. Let t_k be the wall-clock time at loop iteration k, h = `opt.timestep` the physics step, and c the catch-up cap (`MABEL_MAX_CATCHUP`, default 50 ms). The accumulator is

$$a \;\leftarrow\; a + \min\big(t_k - t_{k-1},\, c\big), \qquad \text{then while } a \geq h:\ \big[\,\text{controller.step}(f, j, h);\ \texttt{mj_step};\ a \mathrel{-}= h\,\big], \tag{2}$$

where f is the current adapted frame and j the navigation joystick. Clamping the elapsed time at c is what prevents a latency backlog: without it, one iteration would batch up to half a second of physics chasing a hand pose that is already half a second old, so the robot visibly trails the operator. With a small c the loop returns for a fresh frame every few milliseconds; physics that cannot keep up simply advances in slight slow-motion instead of accumulating lag. Each substep also writes the robot configuration to the published memmap (Sec. 2.4), at the full physics rate (≈ 280 Hz), because a memmap write of $n_q + 1$ doubles costs ≈ 1 µs.

What keeps the accumulator at 1.0×. Eq. (2) holds real-time only while the outer loop runs fast enough to drain a: with $c = 50$ ms each iteration advances at most 50 ms of sim, so the sim tracks wall-clock down to a ≈20 Hz outer loop and no faster loop can build a backlog. Two effects on the development Mac silently pushed the loop *below* that floor and dragged the sim — and therefore *every* client — to a fraction of real-time. Both are now defused, and they are the single most important reason teleop felt "stuck at 8 Hz" before this revision.

LiDAR sensors disabled on the sim path. The `mabel_home` scene carries a 360-beam rangefinder array (a simulated LiDAR for the ROS Nav stack) that ray-casts on *every* `mj_step`, and it dominates the step cost: ≈ 12.7 ms with sensors on versus ≈ 2.0 ms off on this 116-DOF model (0.31× vs. ≈ 2.0× real-time at $h = 4$ ms). Nothing on the plain Mac sim path consumes that `sensordata` — the cameras render in a *separate* process from the `qpos` memmap (§2.4), and the IMU/LiDAR feed only the `+ros` SLAM/Nav path — so the bridge disables sensors whenever the dispatch target is `sim:mujoco`:

```
model.opt.disableflags |= mjDSBL_SENSOR
```

This alone restores the sim from 0.31× to faster-than-real-time (the accumulator then caps it back to 1.0×); the `+ros` path keeps sensors because its SLAM/Nav actually read them.

macOS background throttling (App Nap). A subtler, larger regression: opening the desktop `--viewer` collapsed the *whole* server to ≈5–12 Hz for every client. The cause is macOS power management, not GPU load. The bridge is a window-less *background* process; the moment a *foreground* GUI (the `mjpython` viewer) is frontmost, the OS applies App Nap / timer coalescing to the background process and stretches its `time.sleep` pacing, so the outer loop falls from ≈240 Hz to 5–12 Hz — below the 20 Hz accumulator floor, leaving the sim at ≈0.4× real-time (robot at half speed). The fix is to declare the bridge a user-initiated, latency-critical activity for the whole process lifetime, held in a module global so it is never released:

```
NSProcessInfo.processInfo().beginActivityWithOptions_reason_(
0x00FFFFFF | 0xFF00000000, "...real-time control loop")
```

where `0x00FFFFFF` is `NSActivityUserInitiated` (exempt from App Nap / sudden termination) and `0xFF00000000` is `NSActivityLatencyCritical` (exempt from timer coalescing); *both* flags are required — the first stops the nap, the second keeps the millisecond-scale sleep cadence honest. The validation carried a lesson worth recording: an early measurement on a *contaminated* Mac (a leftover iOS simulator, a camera server, and several stale bridges all competing) read 43 Hz at zero clients and wrongly suggested the guard did nothing; a clean re-measure (Fig. 4) shows it holds 1.0× with the viewer open and the robot moving (bridge ≈300 Hz).

The effort channel must not be gated on the control mode. Under `sim:bridge` (the default target, and the structural twin of the real robot) the gateway holds only a kinematic *mirror*: it never steps physics, and its decision reaches the robot solely through what `SimBridgePlant.advance()` projects onto the RPC — a base twist, servo targets, an external wrench, and the *effort* channel τ read out of `qfrc_applied`. That last projection used to be wrapped in `if wbc_active`, on the reasonable-looking assumption that only the torque modes produce effort. They do not: the controller writes an upper-body gravity feedforward in *every* control space (`whole_body_controller.pdf`, § "Mode-invariant gravity compensation"). The gate therefore made that compensation a fiction in Drive — computed into a model nobody steps, then discarded, so the physics owner saw **0 of**

22 compensated DoFs while Joint/Task saw 14. Every return to Drive sagged the arms, and the next Task entry re-seeded its set-point from the sagged pose: measured 26.5 mm of cumulative right-palm travel over five Drive↔Task cycles, now 0.7 mm. The gate is gone; `wbc_active` survives in the signature for the ROS and in-process plants but no longer decides whether gravity reaches the robot. The lesson generalises to anything this plant computes: on the split deployment, *written into the mirror* and *sent* are different things, and only the second one moves the robot.

The clutch. Teleoperation uses a relative (clutched) mapping: on the first tracked frame, or whenever the operator presses ENTER (or the headset's recalibrate), the `Retargeter` anchors the current hand pose to the current robot pose with no jump, and the IK targets are synced to the present configuration. This decouples absolute operator position from absolute robot position, exactly like lifting and repositioning a mouse.

2.4 Decoupling physics from rendering

On macOS the off-screen MuJoCo `render()` is a *fixed* GPU-readback stall of ≈ 49 ms to 67 ms per camera, *independent of resolution*, because Apple's deprecated-OpenGL path cannot be swapped for `egl`/`osmesa`. Running that on the physics thread would starve the real-time loop. The bridge therefore publishes the robot's configuration q to a memory-mapped file (`/tmp/mabel_teleop_state.f64`, layout $[\,t,\ q_1, \ldots, q_{n_q}\,]$, plus a `.scene` sidecar naming the model), and a separate process, `sim_camera_server.py`, mmaps it read-only, recomputes kinematics only when t changes, renders the cameras, and serves the MJPEG. Physics and video then run on independent cores, exactly like a real robot whose cameras are separate hardware — though the scheduler must still be told to favour the control loop over the renderer (`utility` QoS, Fig. 6). The optional desktop window (`view_robot.py`) is a third reader of the same memmap.

2.5 Weighted camera scheduling

Because the machine produces only ≈ 24 camera renders per second total (shadows off; §3), rendering all cameras equally makes each one a ∼ 8 fps stream. Each camera is instead assigned an integer *weight* w_c — its share of the render budget (head $w = 4$, each wrist $w = 2$, the unused stereo eye $w = 0$). A smooth weighted round-robin (the Nginx algorithm) spreads each camera's slots evenly rather than clumping them: with total weight $W = \sum_c w_c$, every tick adds w_c to each camera's credit, renders the highest-credit camera $c^\star$, and subtracts W from it:

$$\forall c:\ \ \mathrm{credit}_c \mathrel{+}= w_c; \qquad c^\star = \arg\max_c \mathrm{credit}_c; \qquad \mathrm{credit}_{c^\star} \mathrel{-}= W. \tag{3}$$

A render-on-change guard makes a still robot cost *zero* GPU: a camera is never re-rendered for a configuration t it has already captured. The resulting JPEG is pushed to every MJPEG path fed by that camera (e.g. the mono `main` panel reuses the left eye), and each HTTP subscriber holds a one-slot queue so a slow client drops stale frames instead of building a backlog.

2.6 Latest-frame-wins UDP video

MJPEG rides on TCP, whose reliability is the wrong contract for teleop video: one lost segment stalls the *whole* stream until the retransmit lands (head-of-line blocking, ≥ one RTO ≈ 200 ms), and during the stall the socket queues stale frames that then play back late. Since only the freshest frame is worth showing, `udp_video.py` serves the same JPEGs with *drop-don't-queue* semantics: each frame is sent as $\lceil n/1182 \rceil$ datagrams of ≤ 1196 B (clears a Tailscale/WireGuard tunnel, MTU

1280, without IP fragmentation), and the receiver lets a newer frame evict any incomplete older one — a lost chunk costs exactly one frame, never added delay. Clients subscribe with a 1 s JSON keepalive datagram (3 s expiry); the bridge advertises the port in its `hello` (`video.udpPort`, default `:9091`, `--udp-video-port` / `MABEL_UDP_VIDEO_PORT`). The sender installs into both rendering processes through a one-line `PushCapture.on_push` tee, leaving the MJPEG/HTTP machinery untouched as the automatic fallback. The wire format is specified in `teleop_protocol.pdf` (§the video channel); the latency and loss-robustness measurements are in the Vision Pro app document (`vision_pro/docs/visionpro_app.pdf`, Revision 4): clean-link parity with MJPEG (0.35 vs. 0.27 ms loopback mean) and flat 0.3–0.5 ms delivered-frame latency under 1–10% datagram loss.

2.7 Unified H.264: one codec, two transports

JPEG is intra-only: every frame is independent (ideal for drop-don't-queue UDP) but carries no temporal compression, so a crisp 960×720 head costs ≈ 13 Mbps (§3). That is fine on a clean LAN but saturates a weak Wi-Fi or Tailscale link. H.264 adds inter-frame (P-frame) compression for ≈ 5–× fewer bits at the same resolution, so the system now speaks *one codec across every client* (iOS, Vision Pro, web), sim *and* the real robot, differing only in transport:

- **LAN** — H.264 over the same drop-don't-queue UDP (`:9092`), at the camera's *native* render resolution ("match source") with a bitrate that scales with $w\,h$ fps. This is the crisp, low-latency path.
- **WebSocket** — H.264 over a WebSocket (`/camera/<name>/h264`) for browsers, which cannot receive UDP. This profile downscales (long edge → 640 px) and halves the GOP (∼ 0.5 s IDR) so a lossy or reconnecting client recovers quickly. This is the low-latency-over-bandwidth path.

On the Jetson the encoder is hardware `h264_nvenc`; in simulation it is Apple's hardware `h264_videotoolbox` (software `libx264 -tune zerolatency` fallback), selected at runtime (`sim_h264.py`). All backends emit the *identical* wire contract the decoders require: **baseline** profile, `yuv420p` (High-4:4:4 renders black on iOS VideoToolbox / browser WebCodecs), Annex-B with one access-unit delimiter per frame (we split the byte stream on it), and SPS/PPS in-band at every IDR (`dump_extra=freq=keyframe`) so a late or lossy joiner — including a WebSocket reconnect — recovers within one GOP. The bridge advertises the H.264 port in its `hello` (`video.h264Port`, default `:9092`); both apps decode with VideoToolbox (one access unit → `CMSampleBuffer` → `CGImage`) and keep MJPEG as the universal fallback behind a 2.5 s watchdog. The encoder is demand-driven (one `ffmpeg` per watched (camera, profile), reaped after 4 s idle), and on the Mac `videotoolbox` runs on the Media Engine, off the cores the real-time physics loop needs — so an unwatched stream and an idle codec both cost ≈ 0 CPU.

3 Experiments

All numbers below were measured on the development Mac and are regenerated into the figures by `scripts/make_doc_figures.py` (run `make figures`); the harness plots the recorded measurements so the figures and this document cannot drift. The real-time-factor and viewer-contention figures (Figs. 4, 5) are regenerated by `tools/exp_viewer_contention.py`, which records this revision's measurements and offers `--live-rtf`/`--live-fps` to re-probe a running stack. The two quantities that govern user-perceived performance are the per-stream camera frame rate and the physics real-time factor.

The shadow pass dominates the render cost. A single off-screen `render()` is *independent of camera resolution* (measured ≈ 27 ms at 640×480 through 1280×960 alike, shadows off, *idle GPU*; `tools/exp_video_latency.py`), because the cost is a fixed GPU sync/readback on Apple's deprecated-OpenGL path, not pixel work. JPEG encode and decode are an order of magnitude smaller still (< 7 ms combined even at 1280×960 q95), as is the headset's per-frame mipmap regenerate, so *no* downstream stage — encode, wire, decode, texture upload — is the bottleneck. Shadows are: a single shadow-casting light forces a second scene render from the light's viewpoint each frame (≈ 32 ms, 73 vs. 41 ms; the shadow-map *resolution* is irrelevant, 1024 and 4096 both ≈ 70 ms), so the teleop feeds render with shadows *off* by default. The LAN default streams the head at 960×720 per eye and each wrist at 480×360; with the panel now sampling trilinearly off a per-frame mip chain (Sec. 2.6) this is sharp and shimmer-free, and the deprecated default of 800×600 q60 was a workaround for that missing-mipmap shimmer, not a render limit.

The real limit is concurrent-render contention — so render on demand. Resolution-independence holds only for *one* camera on an idle GPU. The server runs one render thread *per* camera, and they share a single GPU whose readback throughput is fixed: N simultaneous renderers each get $\approx 1/N$ of it. Measured concurrent frame rates (`exp_video_latency.py`, Fig. 2): a lone head reaches ≈ 26 fps; stereo head + two wrists (four renderers) drops every stream to ≈ 12 fps. This is why a head bumped to 1280×960 felt "laggy" while a wrist stayed smooth — not a render cliff (there is none on an idle GPU) but the head's larger, doubled (stereo) readback losing the four-way split. The fix is *demand-driven rendering*: a camera with no live subscriber (MJPEG client or UDP sub) renders nothing (`PushCapture.has_clients`, `UDPVideoSender.has_subscriber`). A browser or iPhone watching only the mono `main` eye therefore stops paying to render the headset's right eye and idle wrists, and the head jumps to ≈ 25.6 fps (head only) or ≈ 17 fps (head + both wrists), measured end-to-end over the live MJPEG socket — up from ≈ 14 fps. A Vision Pro that subscribes to both eyes still gets full stereo; nothing renders that nobody is watching.

H.264 vs. JPEG transports (live sim). Measured against a live `run.sh` stack on the `main` head camera (960×720), decoding each transport with PyAV/OpenCV and comparing on the *same* sim (`tools/exp_h264_video.py`, Fig. 3):

transport	resolution	fps	bitrate	KB/frame
H.264 / UDP (LAN)	960×720	23	1.94 Mbps	10.2
H.264 / WS (browser)	640×480	23	1.56 Mbps	8.2
JPEG / UDP	960×720	30	13.0 Mbps	53.8
MJPEG / HTTP	960×720	30	13.25 Mbps	53.2

H.264 carries the *full* 960×720 head at 1.94 Mbps against JPEG's 13.0 Mbps — a 6.7× bandwidth reduction at identical resolution (larger still on a static scene, where P-frames are tiny) — while the per-frame freshness delay stays at parity on a clean link (median 0.5 ms, p95 1.0 ms loopback, all decoders 100%). The loss sweep is the cautionary result that fixes the transport split: at 960×720 a single lost datagram corrupts the H.264 GOP until the next IDR, so under 5% induced loss the high-res UDP H.264 stream collapses while intra-only JPEG degrades gracefully per frame. H.264/UDP is therefore the *LAN* path (where loss ≈ 0); browsers use H.264 over reliable WebSocket/TCP (no loss, downscaled, tight GOP).

Physics real-time factor across configurations. The step cost is *not* a timestep-independent constant: it is dominated by whatever the scene makes `mj_step` compute. With the LiDAR sensors

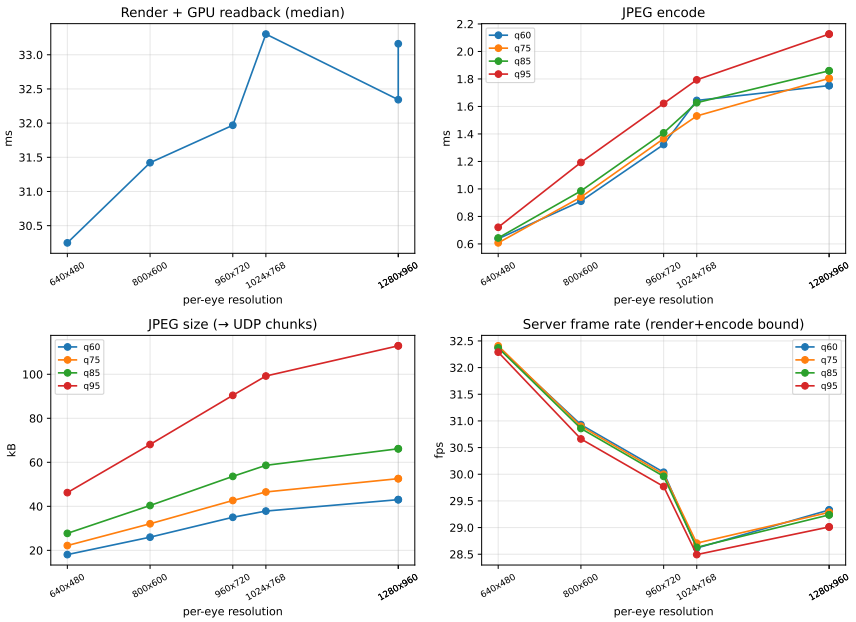


Figure 2: Per-stage video latency vs. per-eye resolution (idle GPU, shadows off; `tools/exp_video_latency.py`). Render+readback is flat ≈ 27 ms across resolutions (top-left) while encode and JPEG size grow gently with resolution and quality (top-right, bottom-left); the single-camera frame rate (bottom-right) is therefore render-bound, not pixel-bound. The operative limit is instead *concurrent* contention (Sec. 3): four simultaneous renderers share one GPU and each falls to ≈ 12 fps, which demand-driven rendering avoids by never rendering an unwatched camera.

active it is ≈ 12.7 ms and the sim runs 0.31× real-time at $h = 4$ ms; with them disabled on the sim path (§2.3) it falls to ≈ 2.0 ms, i.e. a ≈ 2.0× capability that the catch-up accumulator caps back to the 1.0× operating point. Fig. 4 reports the *operating* real-time factor — the value the operator feels — measured straight from the published sim clock (RTF = $\Delta t_{\text{sim}}/\Delta t_{\text{wall}}$, with the robot moving so the viewer renders at its full cap; `tools/exp_viewer_contention.py --live-rtf`) across the six configurations that matter. Both regressions (sensors-on, 0.31; viewer with no App-Nap guard, 0.40) and their fixes (1.00) are visible, as is the residual GPU contention of the full stack with the viewer open (0.88, below). This bounds how fast the *robot moves*, not the video frame rate, which is governed independently by Sec. 2.5.

The desktop viewer is a GPU competitor — cap it. The App-Nap guard fixes *scheduling*, but the viewer (`view_robot.py`) is still a third heavy renderer sharing the one GPU with the camera off-screen renders (§2.4). Its render-on-change loop already drops a still robot to an 8 Hz idle floor, but while the robot moves it renders at `MABEL_VIEWER_FPS`, and each `viewer.sync()` serialises against the camera readbacks. Measured with both camera streams pulled and the robot moving (Fig. 5): at the old 20 fps cap the feeds fell to 14.7/15.8 fps with a p99 inter-frame gap of 185 ms (visibly choppy); closing the viewer restored 24.5/26.5 fps at p99 = 47 ms. Lowering the default cap to 12 fps — a perfectly readable debug window — recovers most of the loss (18.8/20.1 fps, p99 = 67 ms) while physics stays at 1.0×. The recommendation is to close the viewer during active teleop and otherwise leave it at the 12 fps default.

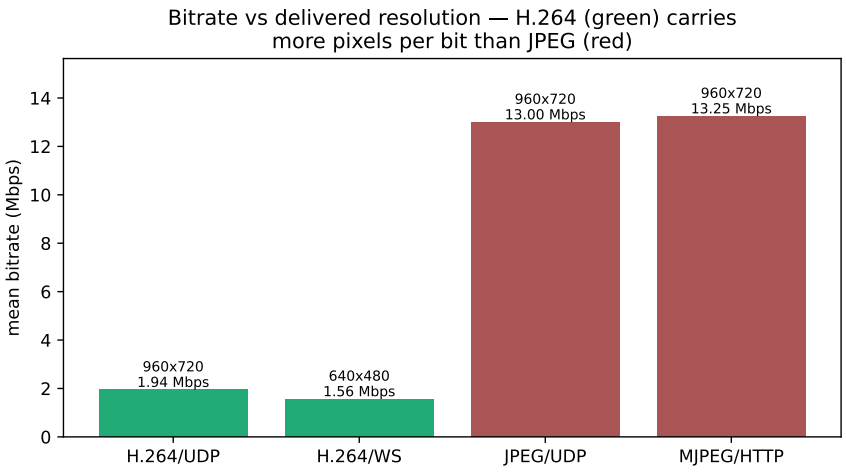


Figure 3: Delivered resolution vs. mean bitrate per transport on the live sim (`tools/exp_h264_video.py`, `main` camera). H.264 (green) delivers the full 960×720 head at 1.94 Mbps; JPEG/MJPEG (red) need ≈ 13 Mbps for the same frame. The browser H.264/WS profile trades resolution (640×480) for a low fixed bitrate and a tight GOP. Companion figures `h264_vs_jpeg_latency.pdf` (freshness-delay parity) and `h264_loss_recovery.pdf` (GOP collapse vs. per-frame JPEG under induced loss) are generated by the same harness.

Process isolation is not enough — the camera server must yield to the loop. Even with physics and rendering in separate processes on independent cores (§2.4), macOS' scheduler does not automatically protect the latency-critical control loop from the camera box. Each camera renders on its own thread, and at *default* scheduling priority those GPU-readback threads contend with the bridge for CPU, dragging the control-loop *iteration* rate — how often a fresh operator frame is consumed and physics catches up to real time — down from its ≈ 160 Hz idle ceiling. Measured with the stereo head and both wrists streaming to five MJPEG pullers while two clients teleoperate in WBC impedance mode (Fig. 6): default priority drops the loop to ≈ 99 Hz with dips to 67 Hz; on a more loaded or thermally throttled machine, with the desktop monitor GUI also up, the same contention is what collapsed it to ∼ 45 Hz and made every client's joint stream visibly lag and desync. Launching the camera server at `utility` QoS (`taskpolicy -c utility`, in `start_teleop.sh`) tells the scheduler to favour the bridge while leaving the cameras on the performance cores: the loop recovers to a steady ≈ 143 Hz at *no* video cost (66.7 vs. 67.5 fps). Background QoS (`taskpolicy -b`, the monitor's launcher) recovers the loop fully but starves video to ≈ 0 fps, so it is deliberately *not* used for the cameras; an explicit `--max-fps 12` cap is a weaker alternative that also costs frame rate. The loop's own compute is *not* the bottleneck: with sensors off a single `mj_step` on this 116-DOF model measures 0.46 ms (≈ 8.6× real-time at the 4 ms step) and the WBC torque law adds only ∼ 0.2 ms per substep — the bridge now *measures and logs* this real-time factor at startup (`tools/exp_camera_contention.py` reproduces the figure) instead of a stale fixed annotation.

3.1 End-to-end teleop loop latency

Where does the operator-perceived delay actually come from? We instrument the *complete* closed loop on the production scene (`tools/exp_loop_latency.py`, $N = 300$ repetitions per stage, run on the development Mac *with the full operational stack live* — bridge, stereo camera server, desktop

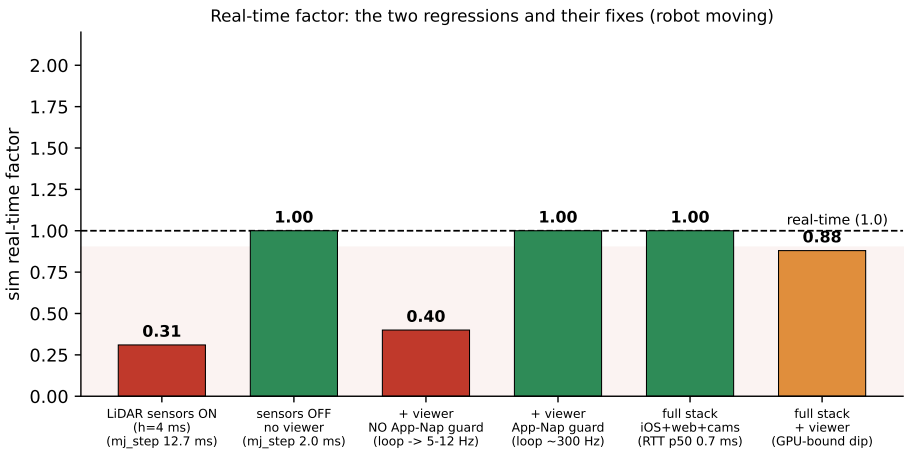


Figure 4: Sim real-time factor (operating value, robot moving) across the configurations that move it, from `tools/exp_viewer_contention.py`. Left to right: LiDAR sensors on (0.31, the "8 Hz server"); sensors off, no viewer (1.00); the desktop viewer open *without* the App-Nap guard (0.40, the background-throttle regression); the same *with* the guard (1.00); the full multi-client stack (iOS app + web + cameras) without the viewer (1.00, app PING→PONG RTT p50 = 0.7 ms); and the full stack *plus* the viewer (0.88, GPU-bound). The dashed line is real-time; the shaded band is below the 0.9 acceptable floor.

viewer — so GPU contention matches real sessions):

$$\underbrace{\text{ARKit est.} \to \text{send}}_{\text{headset (modelled)}} \to \underbrace{\text{net}}_{\uparrow} \to \underbrace{\text{ingest} \to \text{retarget} \to \text{IK} \to \text{physics}}_{\text{bridge (measured)}} \to$$
$$\to \underbrace{\text{render} \to \text{encode} \to \text{UDP}}_{\text{camera box (measured)}} \to \underbrace{\text{net}}_{\downarrow} \to \underbrace{\text{decode} \to \text{display}}_{\text{headset (modelled)}}$$

Method. Every server-side stage is timed on the real code path with synthetic but realistic two-hand frames (10.3 kB wire size, 27 joints/hand). Headset-side stages cannot be timed from the Mac and are *modelled*, labelled (m): a zero-order-hold stream of period T sampled at a random instant contributes age $U(0, T)$, i.e. mean $T/2$, worst case T. The network one-way delay is a scenario parameter taken from measured RTTs: LAN $\approx$ 2 ms RTT and Tailscale $\approx$ 41 ms RTT (direct WireGuard path, measured between the operator's devices with `tailscale ping`, 2026-06-12). The JPEG decode is timed with OpenCV on the Mac as a *proxy* for the headset's ImageIO decoder (same libjpeg-class workload, different silicon).

Per-stage measurements. Mean (p95) over $N = 300$:

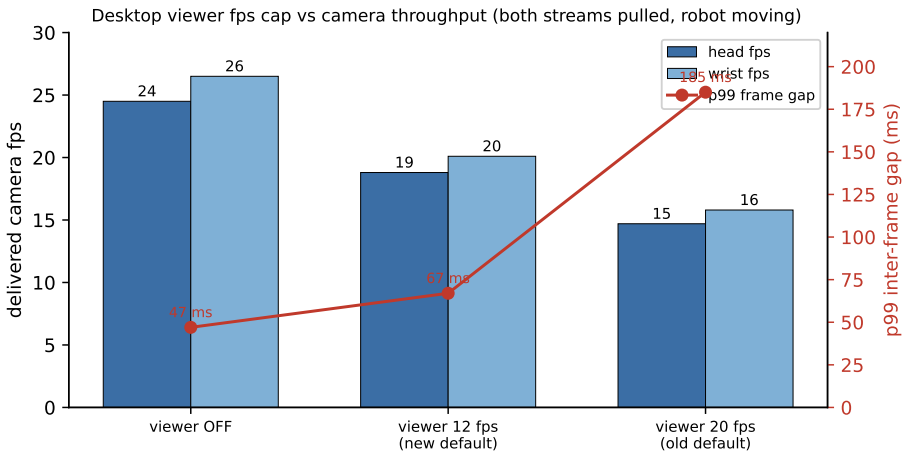


Figure 5: Camera throughput versus the desktop viewer's fps cap (head + wrist pulled together, robot moving; `tools/exp_viewer_contention.py`). Bars are delivered fps per stream; the line is the p99 inter-frame gap. Viewer off is smoothest (≈ 25 fps, p99 = 47 ms); the new 12 fps default recovers most of the loss the old 20 fps cap caused. The physics real-time factor is unaffected by the cap (the App-Nap guard).

Stage	Path	mean (p95) ms
JSON decode + envelope	uplink, per frame	0.12 (0.13)
`decode_teleop_frame` (2 hands)	uplink	0.04 (0.04)
AVP adapter	uplink	0.17 (0.18)
Wrist retarget (SE(3) clutch)	uplink	0.05 (0.06)
Whole-body IK solve (`controller.step`)	per 4 ms substep	0.99 (1.16)
`mj_step` (116-DOF physics)	per substep	3.67 (4.04)
Finger stream apply (cached)	per substep	0.002
GN finger solve (4 it., 16 DOF)	worker @ 20 Hz	5.73 (6.34)
Render + GPU readback (800×600)	return, per slot	76.6 (79.4)
JPEG encode q60 (38 kB)	return	1.4 (1.5)
UDP chunk + send + reassemble	return (loopback)	0.95 (1.16)
JPEG decode (proxy)	return	1.2 (1.4)

Composed budgets. With demand-driven rendering (Sec. 3) the per-eye rate depends on who is watching: a browser/iPhone on the mono `main` eye sees ≈ 25.6 fps (head alone) or ≈ 17 fps (head + wrists), whereas a Vision Pro subscribing to both eyes plus wrists splits the GPU four ways for ≈ 12 fps per eye — *stereo roughly halves the per-eye rate* (Fig. 2). Summing (Figure 7):

Path (mean)	LAN	Tailscale
Command, arm/wrist → actuator target	45 ms	64 ms
Command, fingers → actuator target	76 ms	95 ms
Visual return, sim state → photons	173 ms	193 ms
Glass-to-glass reaction (arm + visual)	218 ms	257 ms

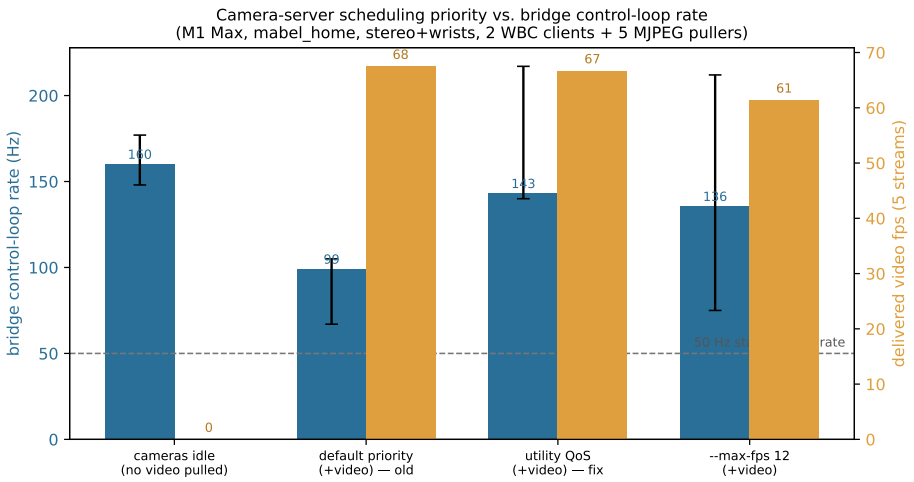


Figure 6: Camera-server scheduling priority governs the bridge control-loop rate, and hence joint-stream lag (`tools/exp_camera_contention.py`; M1 Max, `mabel_home`, stereo head + both wrists pulled by five MJPEG clients, two WBC-impedance teleop clients). Blue bars (left axis) are the bridge loop rate (median, whiskers min–max); orange (right axis) is delivered video fps summed over the five streams. Idle cameras set the ≈ 160 Hz ceiling; *default* priority with video drops it to 99 Hz (dipping to 67 Hz); `utility` QoS restores a steady 143 Hz at unchanged video fps; `--max-fps 12` helps less and costs fps. Server-side `stateAgeMs` stays ≈ 2 ms p50 in all bars because even 99 Hz clears real time on this host — the fix raises the floor so a more loaded machine does not tip below it.

Finding 1: retargeting compute is negligible. The entire server-side mathematics — ingest + SE(3) wrist retarget + whole-body IK + finger GN solve — totals ≈ 12 ms, under 6% of the LAN glass-to-glass time. Moving it to a different computer cannot help (it already runs on the Mac; the headset only streams skeletons), and the hypothesis that hand retargeting causes the perceived lag is *quantitatively rejected*. The finger path's extra 31 ms over the wrist path is the 20 Hz worker cadence ($T/2 = 25$ ms) plus the solve, not the solver itself: raising the worker to 40 Hz would halve that term for 2× the (still small) CPU share.

Finding 2: the visual return dominates. 173 ms of the 218 ms LAN loop is the camera path, and within it 86 ms frame age + 77 ms render stall = 94%. The levers, in order of effect: render *mono* (`--head-only`: head age 25 ms, −96 ms end-to-end, at the cost of stereo depth); a second render process or a machine whose offscreen readback is not ≈ 75 ms fixed (Linux/EGL renders the same camera in ≈ 5 ms to 10 ms — the real robot's onboard camera path will not have this stall at all, the MMlove module delivers H.264/MJPEG directly); accepting the operator-side cost. Network choice is secondary: Tailscale adds only 2×20.5 ms.

Finding 3 (the arm/wrist analysis): servo tracking lag exceeds every transport term. Because the wrist path's command latency (45 ms) is small, the *felt* arm lag must come from dynamics. We step a representative arm actuator by 0.3 rad with all other servos holding pose (Figure 8): the response is first-order with time constant $\tau \approx 228$ ms (arm, $k_p = 100$, $k_d = 20$) and $\tau \approx 140$ ms (wrist, $k_p = 1$), and a gravity-induced steady-state error keeps the 0.3 rad step from ever reaching 90% within 2.5 s. A continuously moving IK target is tracked with phase lag $\approx \tau$ —

Table 1: Measured server characteristics (development Mac, `mabel_home.xml`, 116-DOF model).

Quantity	Value	Source / note
Off-screen `render()`, shadows on	≈ 73 ms	resolution-independent; shadow pass
Off-screen `render()`, shadows off	≈ 41 ms	default; no shadow pass
Head stream, head-only (demand-driven)	≈ 25 fps	whole GPU budget
Head + both wrists (demand-driven)	≈ 17 fps ea.	three-way GPU split
Camera fps, viewer at 12 fps	≈ 20 fps	viewer/camera contention
`mj_step`, LiDAR sensors on	≈ 12.7 ms	rangefinder ray-cast
`mj_step`, sensors off (sim path)	≈ 2.0 ms	sensors disabled; default
RTF capability (sensors off)	$\approx 2.0\times$	capped to $1.0\times$
Operating RTF (robot moving)	$\approx 1.0\times$	Fig. 4; default
`qpos` publish rate	≈ 280 Hz to 310 Hz	per-substep memmap write
State stream, full live stack	50.0 Hz, 0 stalls	app RTT p50 0.7 ms
Input-frame consume policy	newest-only	1-slot deque, stale dropped

so the arm trails the operator's hand by ≈ 0.27 s (τ + command path) regardless of any network or retargeting optimization. The lever here is the *controller*, not the retargeter: stiffer position servos (or computed-torque/impedance control with gravity compensation feed-forward) on the arm chain would cut τ several-fold; on the real robot this is the actuator-gain tuning already planned for the GO-M8010-6 chain.

Summary. Perceived delay decomposes as: arm servo tracking (≈ 230 ms) > visual return (≈ 173 ms, render-bound) > command transport (45 ms to 64 ms, sampling-quantization-bound) $\gg$ all retargeting/IK compute (< 15 ms). Reproduce with `python3 tools/exp_loop_latency.py` (seeded, prints the tables and regenerates both figures).

3.2 Live state-stream regularity and the LAN latency fixes

The loop budget above is an offline, per-stage *composition*; it cannot see an *intermittent* whole-system stall (the operator's report: "the displayed joints freeze, then jump"). To measure what a connected client actually receives, `tools/exp_state_stream.py` attaches as a passive teleop client (it sends only WebSocket pings, never a frame, so it neither claims the driver token nor moves the robot) and records the inter-arrival gaps of the `robot_state` stream, the PING→PONG round-trip, and the server's `latencyMs` field. On loopback the network term is ≈ 0, so the gap statistics isolate *server-side* regularity (event-loop occupancy + the GIL tug with the physics thread).

Baseline (live bridge, loopback, 30 s). The stream is healthy and scales: 44.8 Hz effective (nominal 50), inter-arrival p50/p95/p99 = 22.2/25.4/27.7 ms, **max** 29 ms**, zero stalls**, app RTT 0.6 ms, and *no* degradation from 1 to 8 concurrent clients (44.7–44.8 Hz each). So the per-client re-encode is not a bottleneck and a shared broadcaster would be premature. Two real defects remain, both invisible on loopback and consistent with the LAN/Wi-Fi symptom:

- **Oversized send buffer.** The `websockets` default `write_limit` = 64 KiB holds ≈22 of our 2.98 KB state frames, so a brief Wi-Fi hiccup lets `ws.send` return instantly and *queue* ≈440 ms of stale frames that then replay late. Setting `write_limit` = 8 KiB (≈3 frames) forces back-pressure almost immediately, so the 50 Hz loop blocks and *coalesces to the freshest* state instead of buffering history.

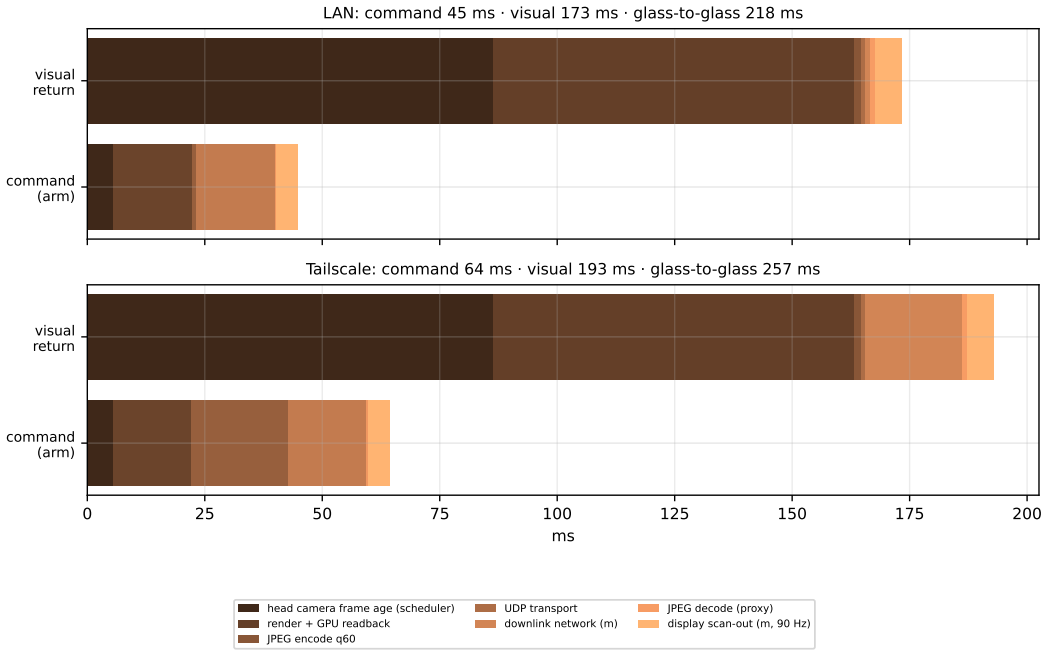


Figure 7: Composed loop-latency budgets, LAN vs. Tailscale. The visual return path (bottom bars) dominates the glass-to-glass time in both scenarios; within it, the macOS render+readback stall and the resulting 5.8 fps head-eye frame age are $\approx 94\%$ of the path. All compute stages together (ingest, retargeting, IK, encode, transport) are under 15 ms.

- **Nagle batching.** The legacy `websockets` server does not set `TCP_NODELAY`; Nagle + delayed-ACK batches the small frames into periodic ∼40–200 ms stalls on real TCP. We now set `TCP_NODELAY` per connection.

Three further server-side fixes. (i) *Phase-accurate pacing*: the old `sleep(0.02)`-then-encode pattern paid 20 ms + work per cycle (measured 40.5 Hz under a synthetic 3 ms send); scheduling each tick at a fixed cadence and sleeping only the remainder restores a true 50.0 Hz. (ii) *Cached joint reads*: `_read_robot_joints`/`_read_base_state` resolved 54 joints by name (`data.joint(name)`) on *every* physics substep — pure-Python work on the control thread holding the GIL from the network thread. Caching the qpos/qvel addresses once is bit-identical and 65.2 µs → 5.5 µs (11.9×) for the joints and 10.8 → 4.2 µs for the base. (iii) *Honest latency metric*: the field the apps displayed as "latency" was the server's `latencyMs` = *time since the last inbound message* (measured 43–243 ms on loopback while the true RTT was 0.6 ms). The apps now display their own WebSocket PING→PONG round-trip (skew-free, transport-inclusive), and the bridge additionally streams `stateAgeMs`, the server-clock age of the published pose — the distinct "the robot itself is lagging" signal that grows only when the control loop falls behind real-time.

Before/after (loopback, 30 s). Applying the fixes and re-measuring confirms the scheduling and read-caching gains directly (the buffer/Nagle fixes only manifest over a real transport): effective rate 44.8 → 50.0 Hz, gap p99 27.7 → 20.4 ms, max 29.3 → 22.1 ms, and jitter 1.3 → 0.2 ms (6.5× tighter); zero stalls throughout. The displayed "latency" is now the app's own 0.5 ms RTT rather than the 43–243 ms command-staleness field.

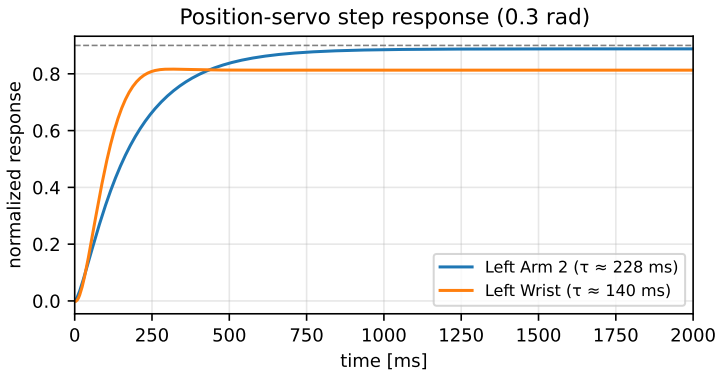


Figure 8: Position-servo step response (0.3 rad, all other joints holding). First-order tracking with $\tau \approx 228$ ms (arm) / 140 ms (wrist); the gravity-load steady-state error of the pure position servo is visible as the asymptote below 1. For teleop, a moving target is followed with phase lag $\approx \tau$ — the largest single latency in the system.

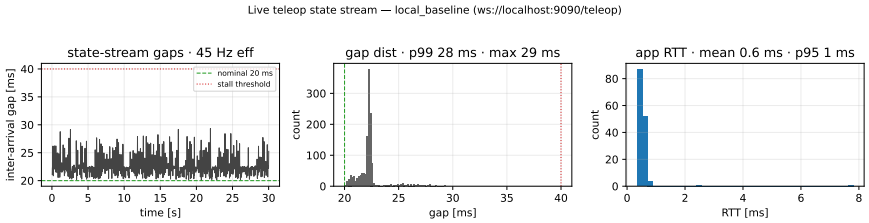


Figure 9: Live `robot_state` stream measured by `tools/exp_state_stream.py` (loopback, 30 s): inter-arrival gaps over time (left), gap distribution (centre), and app PING→PONG RTT (right). The server side is regular (no stalls, p99 = 27.7 ms); the LAN delay the operator sees comes from the buffering and Nagle defects above, which do not manifest on loopback.

End-to-end under the full live stack. The fixes above were re-validated with *everything running at once* — the iOS app (rendering all three camera feeds and the 3D twin), a web teleop client, and the camera server — against the live bridge. The passive probe then records: effective rate 50.0 Hz, inter-arrival p99 = 29 ms, **zero stalls**, and app-level PING→PONG RTT p50 = 0.7–1.4 ms (max ≈2.7 ms). The displayed latency is the app's own *app-level* ping ({`type:ping,payload:{t}`}, echoed by the bridge as {`type:pong,payload:{t,srv}`}) — robust where the protocol-level WebSocket ping had proved unreliable — which replaced the misleading server `latencyMs` (command-staleness) field on all three clients.

Reproduce with `python3 tools/exp_state_stream.py --secs 30` (loopback) or `--url ws://<robot>/teleop --tag wifi` to add the real transport term; `--clients N` runs the scaling test.

4 Implementation and Usage

Dependencies. Python ≥ 3.9 with `websockets`, `aiohttp`, `opencv-python`, `numpy`, and `mujoco` ≥ 3.0 (the development setup uses the Anaconda Python at `~/anaconda3/bin/python3`, which has MuJoCo importable). Install with `pip install -r requirements.txt`.

One-command launch. From `server/`:

```
./run.sh                 # camera box + bridge + control studio, default home scene
./run.sh --viewer        # also open the native Mac MuJoCo window
./run.sh --no-studio     # teleop stack only (skip the web control GUI)
./run.sh -s path/to/scene.xml
```

`run.sh` prints the Mac's LAN IP and the endpoint URLs, advertises the service over Bonjour (`_mabel-teleop._tcp`) so the iOS app discovers it without typing an IP, and wraps `start_teleop.sh`, which starts the camera box (and optional viewer) in the background and the bridge in the foreground. The bridge is foreground on purpose: ENTER re-anchors the clutch and a single CTRL-C tears the whole stack down.

`run.sh` also brings up the Simulation Control Center studio (`web_control_gui/simulation_control_center`, `:8097`) in attach (`--gui-only`) mode and opens it in the default browser once it answers, so every launch hands out a full web control GUI alongside the wire endpoints. The studio never owns the sim in this mode — `run.sh`/`start_teleop.sh` keep physics ownership, and the studio process is reaped when `run.sh` exits. The real-robot counterpart (`server/real_run.sh`) does the same with the real Control Center on `:8094`, reusing the robot's systemd studio service when one is already listening instead of starting a duplicate.

Running by hand (separate logs).

```
# 1) physics + teleop (publishes qpos, stays real-time; --mock for no headset)
python sim_teleop_bridge.py --scene ../simulation/mabel_mujoco/scenes/mabel_home.xml
# 2) the cameras (separate core; reads qpos, serves MJPEG :8080)
python sim_camera_server.py            # --head-only / --head-weight N / --stereo
# 3) (optional) native desktop mirror of the robot
mjpython view_robot.py                 # macOS needs mjpython
```

Passing `--cameras` to the bridge renders in-process (no separate server, but slower physics); the 3-process split is the fast path.

Network setup. Both devices must be on the same Wi-Fi/LAN. The servers bind `0.0.0.0`, so set the client's *Robot Host* to the Mac's LAN IP (e.g. `192.168.1.188`, *not* `127.0.0.1` or a `.local` name on the headset). On the Vision Pro: Settings ▷ Network ▷ Robot Host, then Apply & Reconnect, and grant *Local Network*, *Hand Tracking*, and *World Sensing*. Allow the macOS firewall prompt for `python`. Sanity-check the video from the headset's Safari at `http://<mac-ip>:8080/`, which serves an index page listing every stream and its live/error status. Ports and hosts are overridable via the environment (`MABEL_WS_PORT`, `MABEL_HTTP_PORT`, `MABEL_WS_HOST`, ...; see `config.py`).

Driving from the iPhone. In the MABEL app, Robot tab ▷ Connection ▷ Host = the printed IP ▷ Connect (allow Local Network). The Drive tab in landscape shows the live cameras; the left stick translates the swerve base and the right stick rotates it. Leaving Host = `demo` or blank runs fully offline against the on-device simulator.

Sim→hardware path. The WebSocket + MJPEG contract is hardware-agnostic. To drive the real robot, a robot-side process consumes the same `TeleopFrame`/twist and serves its real ZED and wrist cameras at the same endpoints; the apps connect unchanged. The control math is already shared via `controller/mabel`, so sim and hardware execute identical logic.

5 Networked deployment: ROS stack and live SLAM map

The Mac server (`run.sh`, Section 4) is the laptop development path: pure MuJoCo, no ROS, LAN-only. Two further launchers bring up the *ROS-backed* stack on a Linux/NVIDIA host so the apps additionally receive a live 3D SLAM map. Remote operators reach any of them over Tailscale (the same protocol on the host's tailnet address); nothing is exposed to the public Internet. All three launch the *same* `sim_teleop_bridge.py`, so the wire protocol of Section 2.1 is unchanged; only the data *sources* and the *transport* differ. This section documents those two additions and the complete port map, because they span four machines and silently break if a single port or route drifts.

5.1 The ROS server stack (`sim_ros_server.sh` / `real_ros_server.sh`)

Both scripts source `ros_server_common.sh` and differ only by `MODE` (`sim` vs. `real`). They run **three process groups, each in its own environment subshell**, because the two ROS distributions in play are mutually exclusive and cannot share one shell:

- **Backend** (conda RoboStack ROS, `scripts/mabel_env.sh`). *sim:* `ros2 launch mabel_sim sim.launch.py physics:=false` — camera/LiDAR renderer + `swerve_odometry` (publishes `/odom`) + `robot_state_publisher`, with *no* physics node. *real:* `ros2 launch mabel_bringup hardware.launch.py` — the real HAL and cameras.
- **SLAM** (apt Isaac ROS, `scripts/slam_env.sh`, which wipes the conda env from `PATH`). `ros2 launch mabel_slam visual_slam.launch.py` — cuVSLAM (publishes the `map`→`odom` correction TF) + nvblox (TSDF reconstruction → `/nvblox_node/mesh`). A companion node `mesh_to_pointcloud` re-emits that mesh as a stock-RViz-renderable `PointCloud2` on `/nvblox_node/mesh_pointcloud` (Section 5.2).
- **Server** (conda RoboStack ROS). `sim_teleop_bridge.py` with `MABEL_DISPATCH_TARGET=sim:mujoco+ros` (sim) or `real:ros` (real), plus the camera server. In *sim* the bridge **owns** MuJoCo physics and the `/clock` — byte-identical controller, motion model, WBC torque engine and `mj_step` pacing to the Mac sim — and *additionally* publishes the canonical ROS state (`/clock`, `/joint_states`, `/odom_ground_truth`, IMU) that the sensor renderer, SLAM, and Nav2 compose on top of. There is exactly **one** physics and `/clock` authority (the bridge), which is why `sim.launch.py` runs with `physics:=false`.

A fourth optional group (`mabel_navigation`, Nav2 NavFn $A^\star$ + MPPI) turns web/app map taps into base motion. All groups share one DDS domain: the env scripts force **CycloneDDS on** `ROS_DOMAIN_ID 0`, so the conda sim/server and the apt SLAM interoperate over DDS. Because `nvblox_msgs` is an apt-only package *absent from the conda env*, the bridge's map subscriber falls back to the converted `PointCloud2` rather than the raw `nvblox_msgs/Mesh` (Section 5.2).

5.2 The live SLAM map stream (`/map` and `:9092`)

The apps' *Live SLAM* view renders the robot's nvblox mesh and its odometry pose. `map_stream.py` is the producer: a background ROS subscriber (started by the bridge only when `rclpy` imports and `MABEL_MAP_STREAM=1`) that fans binary frames out to a hub, `MAP_HUB`, which the bridge multiplexes onto two transports. The full path crosses both ROS environments:

```
nvblox_node --/nvblox_node/mesh (nvblox_msgs/Mesh, apt env)--> mesh_to_pointcloud
     `--> /nvblox_node/mesh_pointcloud (PointCloud2, latched) --.
swerve_odometry / HAL --/odom (nav_msgs/Odometry)--------------+--> bridge map_stream
                                                                  `--> MAP_HUB --> /map
```

Wire protocol (binary, little-endian). Two frame types, shared verbatim with the iOS reader `MapStreamLink.swift`:

- **MESH** (`0x01`): `u8 type, u8 flags, u16 pad, u32 count`, then `count`×{`f32 x,y,z; u8 r,g,b,a`} in the ROS `odom` frame. `flags` bit 0 marks the final chunk; bit 1 marks a triangle-soup (surface) mesh, which also carries an index buffer. The mesh is *chunked* at `MAP_CHUNK`= 60000 points (≈ 0.96 MB, under iOS's 1 MiB default WebSocket message limit) and capped at `MAP_MAX_POINTS`= 400000 (uniform subsample beyond that).
- **ODOM** (`0x02`): `u8 type, u8 pad×3, f32 x,y,z, qx,qy,qz,qw`, throttled to `MAP_ODOM_HZ`= 30 Hz.

Two transports, one hub. The bridge serves the map on (i) the `/map` **path of the teleop port** `:9090` — multiplexed alongside `/teleop` — and (ii) a **dedicated port** `:9092` (`MABEL_MAP_PORT`) for direct LAN clients, isolating the multi-MB mesh from the 20 Hz control loop and the video. Both are read-only and gated by the same token as `/teleop`. The hub caches the latest mesh chunks and replays them to a late-joining subscriber, so a client that connects after the map is built sees it immediately. Back-pressure is bounded: each socket has its own 256-frame queue and the oldest frame is dropped when a slow client fills it.

QoS, the silent-failure point. `mesh_to_pointcloud` publishes the cloud **RELIABLE + TRANSIENT_LOCAL** (latched, depth 1) and the bridge subscribes with the *same* profile — a mismatch here (e.g. a VOLATILE publisher against a TRANSIENT_LOCAL subscriber) connects silently and never delivers, the classic "mesh renders blank" bug. Odom uses the permissive **BEST_EFFORT** so both sim and nav publishers match. To avoid burning CPU, `mesh_to_pointcloud` publishes the cloud only while it has a subscriber — the bridge *is* that subscriber, so the stream activates on connect.

6 Limitations

- **Render throughput is platform-bound.** The ≈ 49 ms to 67 ms per-camera readback is a macOS deprecated-OpenGL artifact; on a Linux box with `egl`/`osmesa` the budget is far higher. The weighted scheduler is a mitigation, not a fix — the wrist thumbnails are deliberately a ∼ 3 fps glance.
- **Physics caps robot speed.** With sensors disabled the sim has ≈2× headroom at $h = 4$ ms, capped to 1.0× by the accumulator, so the robot tracks normal motion in real time but still cannot out-run *very* fast hand motion (it then advances in slight slow-motion, bounded lag, rather than building a backlog). Speeding the ceiling up means tuning the model (timestep, solver) in the simulation package, not the server.
- **The desktop `--viewer` is honest GPU contention.** The App-Nap guard keeps *physics* real-time with the viewer open (Fig. 4, bridge ≈300 Hz), but the viewer is a third renderer sharing the one GPU with the camera readbacks, so the full stack (viewer + cameras + a live iOS client) dips the real-time factor to ≈0.88 (mild) and drops the camera feeds (Fig. 5). It is fully usable for debugging — it no longer starves the loop — but close it during active teleop for the smoothest video.
- **Single newest-frame input.** Dropping stale frames minimises latency but means transient packet loss is invisible (by design); there is no frame-history interpolation.
- **Stereo head not wired by default.** The head panel is a single mono `WKWebView` pull; true per-eye stereo (`--stereo` plus `StereoHeadEntity.swift`) is heavier and currently opt-in (see

README_stereo.md).

- **Empty-scene cameras render dark.** mabel_sim.xml is a bare floor; use a furnished scene (mabel_home.xml) for a meaningful view.

References

[1] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," *IEEE/RSJ IROS*, 2012.

[2] Apple Inc., "ARKit hand and world tracking (visionOS)," Developer Documentation, 2024.

[3] W. Reese, "Nginx: the high-performance web server and reverse proxy," *Linux Journal*, 2008. (smooth weighted round-robin scheduling.)

[4] "Motion JPEG over HTTP (multipart/x-mixed-replace)," informal web standard, Netscape, 1995.

[5] A. Millane et al., "nvblox: GPU-accelerated incremental signed distance field mapping," *IEEE ICRA*, 2024.

[6] NVIDIA, "Isaac ROS Visual SLAM (cuVSLAM)," Developer Documentation, 2024.